

 quality gate **passed**  codecov **91%** last commit **repo not found**

PermitParser - Building Permit Data Extraction System

 PYTHON **3.11+**  GOOGLE CLOUD  DOCKER  FASTAPI

 TESTING  PYTEST

Project Overview

PermitParser is a series of custom software tools designed to scrape and parse building project proposals from various municipalities in the greater Montreal area. Each municipality uses a different system and document format, requiring city-specific parsers to extract key information from unstructured sources (HTML, PDFs, etc.) and load it into a structured database.

This structured database is provided to our stakeholder, Munera Intelligence, for use in their downstream data analysis and construction prospecting systems.

Team Members

Name	Student Number	GitHub Username
Minh Huynh	40210039	@vibqetowi
Omar Elmasaoudi	40255123	@Omare04
Asif Ali Khan	40211000	@AsifAliKhan2001
Melissa Rahman	40249231	@mE3lissa
Younes Bouhaba	40205816	@Younesbhb
Alexander El Ghaoui	40200062	@Ghawi25
David Onwionoko	40167358	@d-realblank
Steven Zrihen	40174529	@elshito
Youssef Yacoub	40189020	@Pharo-14
Hudson Lu	40254326	@HudsonLu
Allaye Dicko	40224071	@allaye4
Adib Akkari	40216815	@adssib

Novelty, Innovation, and Disruption

The primary technical challenge is the lack of standardization in public data. Building permit formats, terminology, and access methods are unique to each city. A generic, scalable solution is not feasible.

The innovation in this project lies in developing a robust and repeatable methodology for tackling these heterogeneous data sources:

- **Source Analysis:** For each city, we analyze the specific data source, whether it's a web portal, static documents, or searchable database
- **Custom Parser Development:** We write dedicated scripts for each city, employing necessary tools (web scraping libraries for HTML, regex for text, OCR for scanned PDFs) to accurately extract required data fields
- **Data Schema Mapping:** We define a standardized database schema and write logic to map varied, city-specific extracted data into this consistent format

The project is an exercise in applied data engineering, focused on solving a series of distinct but related "messy data" problems using Python applications running on Google Cloud in Docker containers.

 Documentation

- [Project Wiki](#)
- [Bidding Document](#)

 Technology Stack & Rationale

Core Technologies

Technology	Benefits
Python 3.11+	• Excellent libraries for web scraping (BeautifulSoup, Scrapy) • Strong data processing capabilities (pandas, numpy) • OCR and computer vision support (Tesseract, OpenCV) • Robust HTTP handling (requests)
Google Cloud Platform	• Scalable containerized deployment • Cloud Run for serverless execution • Cloud Storage for data persistence • Cloud SQL for structured databases • Firestore for metadata/indexing • Cloud Tasks for orchestration • Pub/Sub for parallel processing • Cloud Functions for event-driven compute • Cloud Scheduler for scheduled jobs
Docker	• Consistent deployment environments • Isolated parser dependencies • Simplified CI/CD pipelines
FastAPI	• High-performance async API framework • Automatic API documentation • Type hints and validation • Native async/await support

Development Environment

Tool	Configuration
------	---------------

IDE	VS Code with Python extensions
-----	--------------------------------

Data Processing Stack

Component	Tools & Benefits
Web Scraping	• BeautifulSoup for HTML parsing • Scrapy for large-scale scraping • Requests for HTTP operations
Document Processing	• PyPDF2/pdfplumber for PDF extraction • Tesseract OCR for scanned documents • OpenCV for image preprocessing
Data Storage	• PostgreSQL on Cloud SQL • Cloud Storage for raw documents • Firestore for metadata/indexing

Quality Assurance

Category	Tools
Testing	• pytest for unit testing
CI/CD	• GitHub Actions

Architectural Patterns & Features

- **Hybrid Orchestration-Choreography Model:** Cloud Tasks for transactional orchestration, Pub/Sub for parallel choreography
- **Adjacency List Pattern:** Used for hierarchical StructureNode tree in Firestore
- **Firestore TTL (Time-to-Live):** Automated retention policy for WorkUnit records
- **Stakeholder Scraping Module:** Designed for associating stakeholders with projects
- **Cloud Scheduler:** Triggers scraping jobs on a defined schedule

 Getting Started

Prerequisites

- Python 3.11 or higher
- Docker and Docker Compose
- Google Cloud SDK (gcloud CLI)
- Git

Local Development Setup

1. Clone the repository

```
git clone https://github.com/MuneraCappin/PermitParser.git
cd PermitParser
```

2. Set up Python virtual environment

```
python -m venv venv
source venv/bin/activate # On Windows: venv\Scripts\activate
pip install -r requirements.txt
```

3. Set up environment variables

```
cp .env.example .env
# Edit .env with your configuration
```

4. Run with Docker

```
docker-compose up --build
```

Project Structure

```
src/
├─ parsers/      # City-specific parsing modules
├─ models/       # Data models and schemas
├─ api/          # FastAPI application
├─ storage/      # Database and file storage utilities
├─ utils/        # Common utilities and helpers
└─ tests/        # Test suites
```

Testing

Run the test suite:

```
pytest tests/ --cov=src --cov-report=html
```

Deployment

The application is deployed on Google Cloud Platform using:

- **Cloud Run** for containerized API services
- **Cloud Storage** for document storage
- **Cloud SQL** for structured data
- **Cloud Build** for CI/CD

Deploy to production:

```
gcloud builds submit --config cloudbuild.yaml
```